

Multi-Query Optimization for Worst-Case Optimal Joins: Shared Computation in Apache Calcite

Thomas Banghart

Abstract

Cyclic join queries (triangle counting, graph motif search, and multi-hop correlation patterns) are intractable for binary join engines. No join ordering avoids intermediate results that can be exponentially larger than the output. Worst-case optimal join (WCOJ) algorithms solve the single-query problem, but modern analytical workloads require batches of structurally similar cyclic queries, and WCOJ alone does not eliminate the resulting redundant computation. We present the first system to combine WCOJ with multi-query optimization in an open-source, general-purpose SQL framework, implemented as modular extensions to Apache Calcite. Our framework introduces the **Combine** relational operator and **MULTI()** SQL syntax for declarative query batching, together with three cross-query optimizations that exploit WCOJ’s variable-at-a-time structure: identity-based trie caching, frequency-aware sub-expression sharing via spools, and shared-prefix execution through join-variable fingerprinting. On synthetic graph workloads, the system achieves up to 4x speedup over sequential binary joins with per-query amortized cost dropping 9.5x as batch size grows. On TPC-H cyclic self-join patterns, WCOJ enables queries that binary joins cannot complete within a 4 GB heap, achieving 3.1x speedup where feasible. We also characterize the failure modes: on FK-based cycles where binary joins efficiently handle selective closing predicates, WCOJ is 1.5x–3.6x slower, and we explain when each strategy applies.

1. Introduction

Join processing is the most critical operation in relational query evaluation. For decades, relational database management systems (RDBMSs) have relied on *binary join trees*: plans composed of pairwise hash joins, merge joins, or nested-loop joins arranged in a tree where each internal node combines two inputs [1]. For acyclic query topologies (stars, snowflakes, chains), a well-chosen binary join order keeps intermediate results bounded by the input and output sizes. However, for *cyclic* queries, no binary join ordering avoids potentially catastrophic intermediate blowup [2, 3].

Consider the triangle query, a fundamental motif in graph analytics:

$$Q_{\triangle}(a, b, c) \leftarrow R(a, b), S(b, c), T(c, a)$$

A binary plan that first computes $R \bowtie S$ on b produces all 2-hop paths (a, b, c) , which can be $O(|E|^2)$ for graphs with high-degree hub nodes, before filtering with T . The final output (actual triangles) may be orders of magnitude smaller. Worst-case optimal join (WCOJ) algorithms [3, 4, 5] resolve this by processing one variable at a time, intersecting candidate values across *all* participating relations simultaneously. The resulting runtime is bounded by the *AGM bound* [2], the information-theoretic maximum output size given the input cardinalities, rather than by intermediate result sizes.

Independently, *multi-query optimization* (MQO) [6, 7, 8] addresses redundant computation when multiple queries share common sub-expressions or perform structurally identical joins. MQO has

been studied extensively for binary-join workloads, but WCOJ’s variable-at-a-time execution creates qualitatively different sharing opportunities that binary-join plans cannot exploit: trie indices can be shared across queries, identical backtracking search trees can be traversed once, and a shared variable prefix can be computed once and distributed to per-query suffix executors. These opportunities remain unexplored.

In this paper, we present an integrated system within Apache Calcite [9] that combines WCOJ execution with multi-query optimization. Our contributions are:

1. **WCOJ in Calcite.** We show that hash-based WCOJ can be integrated into a Volcano-based optimizer without pre-sorted indices or schema changes, via automatic detection of cyclic join subgraphs. The result is an opt-in physical operator usable by any of the systems built on Apache Calcite.
2. **The `Combine` operator and `MULTI()` syntax.** We introduce `Combine`, a relational algebra operator that holds N independent sub-queries as a single multi-root structure, and `MULTI()`, a SQL extension for declaring query batches. `Combine` makes cross-query sharing opportunities visible to the optimizer that are invisible when queries are optimized in isolation.
3. **Cross-query optimizations.** We design three optimizations that exploit WCOJ’s variable-at-a-time structure in ways unavailable to binary-join planners: identity-based trie caching (build each hash trie once across the batch), frequency-aware sub-expression spooling (materialize shared sub-trees once), and shared-prefix execution (traverse the shared WCOJ search space once and distribute bindings to per-query suffix executors).
4. **Formal analysis and experimental evaluation.** We prove correctness of prefix sharing and analyze the cost model, then evaluate on synthetic graph workloads and TPC-H cyclic join queries. The evaluation characterizes both the speedups and the failure modes. FK-based cycles where binary joins are already efficient expose a cost model gap that we quantify and explain.

We also identify the limits of WCOJ’s applicability: on FK-based cyclic queries where a selective closing predicate eliminates most candidates early, binary joins outperform WCOJ by 1.5x–3.6x. A key result is that the value of the `Combine` framework lies not in universally applying WCOJ, but in providing a platform for *selective* optimization, routing cyclic sub-queries to WCOJ where it helps while preserving binary join efficiency where it does not.

2. Background and Related Work

This section surveys the two bodies of work our system bridges (worst-case optimal join algorithms and multi-query optimization) as well as the query processing frameworks that provide the architectural substrate for integration.

2.1 Worst-Case Optimal Join Algorithms

The theoretical foundations for WCOJ algorithms were established by Atserias, Grohe, and Marx [2]. Their *AGM bound* shows that for a join query Q over relations $R_1, \dots, R_n$, the maximum output size is:

$$|Q(D)| \leq \prod_{i=1}^n |R_i(D)|^{x_i^*}$$

where $\mathbf{x}^*$ is the optimal solution to the *fractional edge cover* linear program over the query hypergraph. This bound is tight: for every query and set of cardinalities, there exists a database instance achieving it. For the triangle query, this yields $|Q_\Delta| \leq |R|^{1/2} \cdot |S|^{1/2} \cdot |T|^{1/2} = |E|^{3/2}$, which is strictly better than the $O(|E|^2)$ intermediate result possible with binary joins.

Ngo, Porat, Re, and Rudra [3] proved that **Generic-Join** achieves this bound:

```

GENERIC-JOIN(Relations R_1...R_n, Variables x_1...x_m):
  if m = 0:
    emit matching tuples
    return
  candidates <- intersection of pi_{x_1}(R_i | current bindings)
                    for all R_i mentioning x_1
  for each value v in candidates:
    bind x_1 <- v
    GENERIC-JOIN(R_1...R_n, x_2...x_m)

```

The algorithm processes variables one at a time, computing the intersection of candidate values across all relations that constrain each variable. Their proof proceeds by induction on the number of variables, using the AGM bound at each level to bound the work done.

Subsequent work extended the theoretical landscape: Ngo, Re, and Rudra [18] unified cardinality-based and degree-based bounds; Gottlob, Leone, and Scarcello [21] showed that queries with bounded hypertree width are tractable; and Ngo [24] surveyed open problems including adaptive variable ordering and inequality predicates. Our system uses the AGM bound and Generic-Join; these extensions inform future directions but are not directly employed.

2.2 Practical WCOJ Implementations

Translating worst-case optimal algorithms from theory to practice has been a decade-long effort, with several distinct architectural approaches.

Sorted trie approaches. Veldhuizen’s *Leapfrog Triejoin* [4] proved that WCOJ is practical, but requires pre-sorted tries. This constraint limits deployment to systems like LogicBlox where data arrives pre-sorted. Our hash-based approach eliminates this requirement, enabling WCOJ on general SQL inputs without index pre-computation.

Graph-specialized engines. EmptyHeaded [14] achieves order-of-magnitude speedups over binary-join engines on graph pattern queries using SIMD-accelerated sorted-trie intersection and a GHD-based variable ordering compiler. But it inherits the pre-sorted index requirement and does not support general SQL workloads. Our system addresses both gaps.

Hash-based integration into general-purpose RDBMS. Freitag et al. [5] showed that hash-based WCOJ can be built inside a general-purpose RDBMS, using tries constructed at query execution time with no pre-sorting. Their Umbra integration is the closest precursor to ours; we adopt the same hash-trie approach, but their system processes each query independently. The multi-query dimension is entirely absent.

Unified approaches. Free Join [15] unifies binary and WCOJ under a single framework using a *free join trie* that generalizes both hash tables and sorted tries. Like Umbra, it optimizes individual queries without considering cross-query sharing.

Table 1. Comparison of WCOJ implementations.

System	Index Type	Pre-built?	General SQL?	Hybrid w/ Binary?	Multi-Query?
Leapfrog Triejoin [4]	Sorted trie	Yes	No (Datalog)	No	No
EmptyHead [14]	Columnar trie	Yes	No (graph)	No	No
Umbra [5]	Hash trie	No	Yes	Yes	No
Free Join [15]	Free join trie	No	Yes	Yes (unified)	No
This work	Hash trie	No	Yes	Yes	Yes

2.3 Multi-Query Optimization

Multi-query optimization (MQO) seeks to reduce redundant computation when processing multiple queries. The problem was formalized by Sellis [6] and Finkelstein [23], who established MQO as a *selection problem*: choosing which intermediate results to materialize. The general selection problem is NP-hard; Kathuria and Sudarshan [7] achieved a $(1 - 1/e)$ approximation via submodular maximization, and Jindal et al. [25] demonstrated its importance at datacenter scale. Zinchenko and Ponomaryov [10] survey the broader landscape. Our work does not address this selection problem; we instead exploit WCOJ’s structure to eliminate redundancies without a combinatorial selection step.

Operator-based approaches. Roy et al. [8] showed MQO can be implemented as a Volcano extension for binary-join plans. Tu et al. [11] proposed ψ -operators that algebraically combine multiple queries (up to 36x speedup); our **Combine** operator is similar in spirit but composes with WCOJ rather than binary joins. QPipe [22] shares work dynamically at runtime; our compile-time approach is less adaptive but cheaper to execute. In industry, Bruno et al. [26] and Roy et al. [27] implement computation reuse at scale, both exclusively on binary-join plans.

2.4 Query Optimization Frameworks

Modern cost-based optimizers descend from two foundational systems. **System R** [1] introduced dynamic programming for join ordering, cost-based plan enumeration, and the separation of logical and physical plan spaces. Graefe’s *Volcano optimizer generator* [19] introduced rule-based plan transformation with a top-down, goal-directed search strategy; the *Cascades framework* [20] refined Volcano with lazy evaluation and memoization. Apache Calcite [9] implements a hybrid Volcano/Cascades optimizer and serves as the query processing backbone for Apache Hive, Apache Flink, Apache Druid, Trino, and numerous other systems. Calcite provides a relational algebra with extensible operators (**RelNode** hierarchy), a cost-based optimizer (**VolcanoPlanner**), an enumerable code-generation backend via Linq4j, and a pluggable SQL parser built on JavaCC. Our extensions leverage all four components while preserving backward compatibility.

Tian [13] identifies three key trends in industrial query optimization: tighter feedback loops between optimization and execution, expansion from single-query to workload-level optimization, and composable architectures that enable cross-engine collaboration. Our work directly addresses the second trend by introducing workload-level optimization for WCOJ queries within the composable Calcite architecture.

2.5 The Unexplored Intersection

Despite the maturity of both WCOJ algorithms and MQO techniques, no prior work has combined them. This gap is surprising because the structure of WCOJ execution (trie-based indexing, variable-at-a-time search, and backtracking) creates sharing opportunities that are qualitatively different from those in binary-join workloads:

1. **Trie sharing.** Two triangle queries over the same graph build identical hash tries on edge relations R, S, T . These tries can be constructed once and shared. Binary join plans build flat hash tables whose build/probe sides vary by query, making cross-query reuse structurally impossible.
2. **Search-space sharing.** Consider two queries $Q_1(a, b, c)$ and $Q_2(a, b, d)$ over the same triangle, differing only in which output column they project. Both enumerate *identical* variable bindings for a and b before diverging at the third variable (c vs. d). A binary join plan for Q_1 produces a materialized intermediate $R \bowtie S$ that Q_2 cannot reuse, because the plan structure differs once projections change. WCOJ’s backtracking search, by contrast, produces the same partial bindings ($a = v_1, b = v_2$) for both queries up to the divergence point.
3. **Variable-level factoring.** Because WCOJ processes one variable at a time, a shared prefix ($v_1, \dots, v_K$) can be computed once and dispatched to per-query suffix executors that handle only the divergent variables. Binary joins operate at the tuple level and have no analogous factoring: each plan produces a full tuple before any result is available to share.

Our system exploits all three opportunities through the cross-query optimizations described in Sections 5 and 6.

3. WCOJ Integration in Calcite

3.0 System Overview

The system adds five new components to Calcite’s standard pipeline:

[PLACEHOLDER: Need to make a diagram]

3.1 Operator Design

Our WCOJ operator takes $N \geq 3$ inputs and processes them simultaneously via multi-way intersection, unlike binary join operators that combine exactly two inputs at a time. We implement this as `EnumerableWCOJ`, a physical operator in Calcite’s enumerable convention.

Join Variables. The operator is parameterized by a list of `JoinVariable` objects, each representing an equivalence class of columns across inputs. For the triangle query with inputs $R(a, b), S(b, c), T(c, a)$:

- $v_0 = \{(R, a), (T, a)\}$: the shared variable a
- $v_1 = \{(R, b), (S, b)\}$: the shared variable b
- $v_2 = \{(S, c), (T, c)\}$: the shared variable c

3.2 Cyclic Query Detection

Detecting cyclic join patterns requires flattening the binary join tree so all predicates are visible simultaneously, then testing the resulting join graph for cycles.

Calcite represents joins as a binary tree of `LogicalJoin` nodes, where each node carries only its local condition. Cyclicity is not detectable at any single node. `JoinToMultiJoinRule` (a standard Calcite rule) collapses the inner-join subtree into a single `MultiJoin` with N inputs and one combined equi-join condition, making the global structure accessible. From this combined condition, we extract equi-join predicates and group co-equated fields into equivalence classes using Union-Find. Each equivalence class spanning two or more inputs becomes a `JoinVariable`, a hyperedge in the join hypergraph. The implementation handles general hyperedges of any degree; all queries in our evaluation happen to be degree-2. For cyclicity testing, each hyperedge is expanded into a clique of pairwise edges, and the standard cyclical test is applied: a connected graph with $|E| \geq |V|$ contains at least one cycle. This test fires `EnumerableWCOJRule` and suppresses it for acyclic queries where binary join ordering is already optimal.

3.3 Multi-Level Hash Tries

Our WCOJ implementation uses a purpose-built `HashTrie`: a recursive hash map where each level corresponds to a join variable in the global variable ordering:

`HashTrie` for $R(a, b)$:

```
root
+-- a=1 --> { b=2: [row(1,2)], b=5: [row(1,5)] }
+-- a=2 --> { b=3: [row(2,3)] }
+-- a=3 --> { b=1: [row(3,1)] }
```

The trie supports three operations:

- **build(source, extractors)**: Scans the input once, inserting each row by extracting keys at each level. Time: $O(|R| \cdot d)$ where d is the number of levels.
- **getKeysAtLevel(level, prefix)**: Navigates using the prefix of prior bindings and returns distinct keys at the target level. Time: $O(d)$ navigation + $O(|\text{keys}|)$ enumeration.
- **probe(keyValues)**: Full probe with all levels specified, returns matching rows. Time: $O(d)$.

Unlike Veldhuizen’s sorted trie approach [4], hash-based tries do not support sorted enumeration (which Leapfrog Triejoin uses for its leapfrog intersection primitive). This is an acceptable tradeoff for our setting: we perform hash-based intersection rather than sorted-merge intersection, following Freitag et al.’s insight [5] that hash-based structures can be built during query execution without pre-sorted input or persistent indices.

3.4 WCOJ Enumerator

The `WCOJEnumerator` implements iterative deepening with backtracking over the global variable ordering. The core loop:

```

moveNext():
  Phase 1 (initialization with backtracking):
    level <- 0
    while level >= 0 and level < numVariables:
      initCandidatesAtLevel(level)
      if advanceAtLevel(level):
        level++
      else:
        level-- // backtrack
        while level >= 0:
          if advanceAtLevel(level):
            level++
            break
          level--
    if level < 0: finished <- true; return false
    // Phase 1 has found the first complete binding (level == numVariables).

  Phase 2 (yield + backtrack):
    loop:
      collectMatches() --> yield cross-product of matching rows
      currentLevel--
      while currentLevel >= 0:
        if advanceAtLevel(currentLevel):
          propagate forward to deeper levels
          break
        else:
          currentLevel-- // exhausted, backtrack further

```

The backtracking in Phase 1 is essential for correctness: a candidate value at level k may have no valid continuations at level $k + 1$ even though other candidates at level k do. A naive linear initialization that gives up on the first failure would miss valid results on certain data distributions.

The critical method is `initCandidatesAtLevel(level)`, which implements the intersection step from Generic-Join:

$$\text{candidates}(x_k) = \bigcap_{R_i \ni x_k} \pi_{x_k}(\sigma_{x_1=v_1, \dots, x_{k-1}=v_{k-1}}(R_i))$$

Each trie lookup navigates using the prefix of already-bound variables relevant to that specific input, returning only values consistent with all prior bindings. The intersection across inputs ensures only values satisfying *all* join conditions survive.

3.5 Cost Model

The cost of `EnumerableWCOJ` has a lower bound of:

$$C_{\text{WCOJ}} \geq C_{\text{build}} + C_{\text{enum}} = \sum_{i=1}^n |R_i| + |Q(D)|$$

where C_{build} accounts for trie construction (linear scan of each input) and C_{enum} accounts for result enumeration. This lower bound captures the two unavoidable costs (reading each input once and producing each output tuple) but omits the per-level intersection cost of the WCOJ search:

$$C_{\text{search}} = \sum_{k=1}^m \sum_{(v_1, \dots, v_{k-1}) \in \text{valid}} \min_{R_i \ni x_k} |\pi_{x_k}(\sigma_{x_1=v_1, \dots, x_{k-1}=v_{k-1}}(R_i))|$$

This search cost is the work WCOJ performs exploring candidate values at each variable level. When it is small relative to $C_{\text{build}} + C_{\text{enum}}$ (as in self-join cyclic queries where the output is large relative to the search space), the lower bound is a useful approximation. When C_{search} dominates (as in FK-based cycles where a variable has high fan-out but the query’s final selectivity is low), the lower bound substantially underestimates the true cost, and WCOJ can be slower than binary joins that handle the selective closing predicate efficiently.

We deliberately present this as a lower bound rather than a complete cost model. Accurately estimating C_{search} requires per-level cardinality estimates that our current implementation does not maintain. Section 7.2.4 provides empirical evidence for when the lower bound is tight (self-join cycles, Tables 5–6) and when it breaks down (FK cycles, Table 6), identifying the structural conditions (high fan-out at intermediate variables with low final selectivity) that cause the gap.

4. The Combine Operator and Multi-Query Framework

4.1 Motivation

Even with WCOJ, a batch of N structurally similar queries builds N identical trie structures and traverses the same backtracking search space N times. For a 20-query batch over the TPC-H lineitem triangle, this means building the same three hash tries 20 times and redundantly scanning 6 million rows per trie. The **Combine** operator makes these redundancies visible to the optimizer by presenting all N queries as a single multi-root plan. Importantly, **Combine** is agnostic to the join strategy used by individual sub-queries: a sub-query can be executed with WCOJ, binary joins, or any other physical operator. This makes **Combine** a natural platform for hybrid optimization, routing each sub-query to the strategy best suited to its join structure. The experimental results in Section 7.2.4 confirm this design goal.

4.2 SQL Extension: MULTI()

We extend Calcite’s SQL parser with a **MULTI** construct that declares a batch of queries for joint optimization:

```
MULTI(
  (SELECT e1.src, e1.dst, e2.dst
   FROM edges e1, edges e2, edges e3
   WHERE e1.dst = e2.src AND e2.dst = e3.src AND e3.dst = e1.src),
  (SELECT e1.src, e2.src, e3.src
   FROM edges e1, edges e2, edges e3
   WHERE e1.dst = e2.src AND e2.dst = e3.src AND e3.dst = e1.src),
  ...
)
```


The parser recognizes `MULTI` as a new keyword and produces a `SqlCall` with `SqlKind.MULTI`. During SQL-to-RelNode conversion, each sub-query is independently converted to a relational expression, and all are wrapped in a single `Combine` node.

4.3 The Combine Relational Operator

`Combine` is a new `AbstractRelNode` in Calcite’s relational algebra that holds N independent sub-queries as children. Its key design properties:

Row type. A single `Combine` output row contains all N result sets: each field is the list of results from one sub-query. Formally:

$$\text{rowType}(\text{Combine}) = \text{STRUCT}\langle \text{EXPR}\$0 : \text{ARRAY}\langle \tau_0 \rangle, \dots, \text{EXPR}\$(N-1) : \text{ARRAY}\langle \tau_{N-1} \rangle \rangle$$

where τ_i is the row type of the i -th child. This encoding fits Calcite’s existing type system while preserving the independent-result-set semantics.

Cost model. `Combine` itself has minimal self-cost ($\sum_i |R_i| \times 0.01$ CPU). The optimizer evaluates the cumulative cost through children, allowing optimization rules to improve individual queries or exploit cross-query sharing.

Relationship to existing operators. Standard Calcite has no multi-root operator. The closest analog is `UNION ALL`, but `Combine` preserves independent result sets without requiring compatible schemas. This is essential for MQO: the optimizer can see all queries simultaneously and identify sharing opportunities that are invisible when queries are optimized in isolation.

4.4 Physical Implementation

`EnumerableCombine` implements code generation for the `Combine` operator. During `implement()`, it:

1. Creates a shared `TrieCache` instance and stores it on the `EnumerableRelImplementor`
2. Visits each child, converting each `Enumerable` result to a `List`
3. Packs all lists into a single struct row
4. Returns a singleton `Enumerable` containing that struct

The `TrieCache` creation in step 1 is the critical bridge for cross-query optimization: it provides a shared context that child WCOJ operators use to avoid redundant trie construction.

5. Cross-Query Optimizations

When multiple WCOJ queries execute within a `Combine`, three complementary optimizations eliminate redundant work:

Optimization	Scope	Enabled by
Trie caching	Build each trie once	Always active in <code>Combine</code>
Sub-expression sharing	Materialize shared sub-trees once	Planner rule (opt-in)

Optimization	Scope	Enabled by
Prefix sharing	Compute shared search prefix once	Planner rule (opt-in)

Trie caching is always active when the **Combine** operator is used. Sub-expression sharing and prefix sharing are additive planner rules that can be enabled independently.

5.1 Trie Caching

Problem. Each WCOJ operator independently builds a hash trie from its inputs. When two WCOJ operators join the same relation on the same key, they build identical tries.

Solution. A per-execution trie cache maps each `(input, keyIndex)` pair to a single shared trie. On first access, the cache builds the trie; subsequent requests for the same input and key return the cached instance. The lookup uses object identity rather than structural equality, since comparing the contents of a streaming input may be expensive or impossible.

```
TRIE-CACHE-GET(input, keyIndex):
  if (input, keyIndex) not in cache:
    cache[(input, keyIndex)] <- BUILD-TRIE(input, keyIndex)
  return cache[(input, keyIndex)]
```

Trie cache sharing has two distinct scopes, and the difference directly explains the experimental gap between **Combine** and **Combine-Share** modes:

1. **Intra-operator sharing** is always active. Within a single WCOJ operator, self-joins reference the same input multiple times. The cache builds one trie per `(input, keyIndex)` pair, avoiding redundant construction within a query.
2. **Cross-operator sharing requires spooling.** Without sub-expression sharing (Section 5.2), sibling WCOJ operators within a **Combine** independently scan their inputs, producing separate objects. The identity-based cache will not match across operators in this case. Cross-operator sharing requires **Combine-Share** mode, which materializes shared sub-expressions so multiple operators read from the same object.

5.2 Sub-Expression Sharing via Spools

Problem. When the batch contains structurally identical sub-queries (e.g., the same triangle join appearing multiple times with different projections), each independently builds identical trie structures and traverses the same search space.

Solution. A planner rule detects structurally identical sub-trees across **Combine** inputs via frequency analysis, identifying sub-trees that appear at least twice. The first occurrence is wrapped in a spool (which materializes and caches the result); subsequent occurrences are replaced with spool reads:

Before:	After:
Combine	Combine
<code>+++ Project(a,b,c) <- WCOJ(...)</code>	<code>+++ Project(a,b,c) <- SPOOL(WCOJ(...))</code>
<code>+++ Project(x,y,z) <- WCOJ(...)</code>	<code>+++ Project(x,y,z) <- WCOJ(...)</code>
<code>+++ Project(a,b,c) <- WCOJ(...)</code>	<code>+++ Project(a,b,c) <- READ_SPOOL</code>

The rule skips leaf table scans and the **Combine** root, focusing on higher-level sub-trees where materialization benefit is greatest. Leaf scans are skipped because trie caching (Section 5.1) already eliminates redundant trie construction from shared inputs. Spooling the scan itself would add materialization overhead without additional benefit.

Memory considerations. Spooling materializes intermediate results in memory. For the TPC-H self-join triangle at SF=0.01, each spool holds up to 2.9 million rows (~100–200 MB depending on projection width). The current implementation does not spill spools to disk; disk-backed spooling is a natural extension for production use.

5.3 Shared-Prefix Execution

Problem. Two WCOJ operators may enumerate the same variable prefix identically. For example, two triangle queries over the same graph might share all three join variables and differ only in their output projections. Without sharing, both operators independently traverse the same backtracking search space.

Solution. Detect shared prefixes at compile time via fingerprinting, compute the prefix once at runtime, and distribute the bindings to per-query suffix executors.

5.3.1 Join Variable Fingerprinting To compare variables across different WCOJ operators, we need a canonical representation that is independent of operator-local input numbering. Each variable is fingerprinted using the structural digest of each participating input and its field index:

$$\text{fingerprint}(v) = \text{sort}(\{(\text{digest}(R_{i_k}), f_k) \mid (i_k, f_k) \in v.\text{occurrences}\})$$

Two fingerprints are equal when they represent the same key intersection over structurally identical inputs. This invariant makes prefix detection correct: if fingerprints match at positions $0, \dots, K-1$, the WCOJ backtracking search over those variables produces identical bindings.

5.3.2 Prefix Group Detection Given N WCOJ operators, we build a *trie of fingerprint sequences* to find shared prefixes:

```
Input: WCOJ_0 with variables [fp_A, fp_B, fp_C]
      WCOJ_1 with variables [fp_A, fp_B, fp_D]
      WCOJ_2 with variables [fp_X, fp_Y]
```

Prefix trie:

```
root
  +--- fp_A --> fp_B --> +--- fp_C (WCOJ_0)
  |                      +--- fp_D (WCOJ_1)
  +--- fp_X --> fp_Y      (WCOJ_2)
```

Result: PrefixGroup(depth=2, members=[0, 1])

A traversal of this trie finds *divergence points* (nodes with multiple children) and groups all descendant queries. Groups with depth ≥ 1 and $|\text{members}| \geq 2$ represent opportunities for shared-prefix execution.

5.3.3 Two-Phase Execution Once prefix groups are detected, grouped WCOJ operators are replaced with a two-phase execution strategy:

Phase 1: Prefix computation. The standard WCOJ algorithm is run truncated at depth K (the shared prefix depth), yielding variable *bindings* rather than result rows:

WCOJ-PREFIX($R_1 \dots R_n, x_1 \dots x_K$) : for each valid $(v_1, \dots, v_K)$, yield $(v_1, \dots, v_K)$

Phase 2: Suffix execution. For each prefix binding, a per-query suffix executor sets $x_1 = v_1, \dots, x_K = v_K$, initializes suffix variables $x_{K+1}, \dots, x_m$, and backtracks only within the suffix. A floor parameter prevents backtracking below the prefix boundary:

```
MOVE-NEXT-SUFFIX(floor):
  while currentLevel >= floor:
    if advanceAtLevel(currentLevel):
      propagate forward to deeper levels
    else:
      currentLevel <- currentLevel - 1
  if currentLevel < floor: return false // prefix exhausted
```

The `floor` parameter prevents backtracking below the prefix boundary, confining suffix execution to the per-query divergent portion of the search space.

6. Formal Analysis

6.1 Correctness of Prefix Sharing

Theorem 1. *Let Q_1 and Q_2 be two WCOJ queries with variable orderings $(x_1, \dots, x_m)$ and $(x_1, \dots, x_K, y_{K+1}, \dots, y_{m'})$ respectively, such that (a) variables $x_1, \dots, x_K$ have identical fingerprints at the same positions (i.e., $\text{fingerprint}(Q_1.x_k) = \text{fingerprint}(Q_2.x_k)$ for $k = 1, \dots, K$), (b) the corresponding inputs are shared via *CombineSharedComponentsRule* so that *TrieCache* returns identical trie objects for corresponding variables, and (c) both operators use the same global variable ordering for positions $1, \dots, K$. Then the prefix bindings $\{(v_1, \dots, v_K)\}$ computed by Q_1 's prefix enumerator are exactly the prefix bindings that Q_2 's full enumerator would produce for its first K variables.*

Variable ordering invariant. Condition (c) requires that the two WCOJ operators assign the same variables to the same positions in their global ordering. `WCOJPrefixAnalyzer` (Section 5.3.2) enforces this by comparing fingerprint sequences *positionally*: a prefix group is formed only when the fingerprint at position k in one operator equals the fingerprint at position k in the other, for all $k \leq K$. Since each fingerprint uniquely identifies which inputs and fields participate in a variable's intersection (Section 5.3.1), positional fingerprint equality implies that both operators perform the same intersection at each level k of the search tree, which is exactly what the proof requires. In the current implementation, all WCOJ operators within a `Combine` are constructed by the same `EnumerableWCOJRule` application, which extracts variables from structurally identical join conditions; the prefix analyzer then verifies that the resulting orderings match position-by-position before forming a group.

Proof sketch. The WCOJ execution forms a search tree where level k branches on all valid values of x_k given the partial binding $(v_1, \dots, v_{k-1})$. The candidates at level k are:

$$\text{candidates}(x_k \mid v_1, \dots, v_{k-1}) = \bigcap_{R_i \ni x_k} \text{trie}_i.\text{getKeys}(k, \text{prefix}_i(v_1, \dots, v_{k-1}))$$

By condition (c), both operators process the same variable at each position $k \leq K$. By condition (a), the fingerprints match, so the participating inputs are identical at each level. By condition (b), **TrieCache** returns the same trie objects for corresponding inputs (via identity-based lookup on spooled inputs; see Sections 5.1–5.2). Given shared tries, the candidate sets are identical at each prefix level. By induction on k : identical candidates at level k means the same branches are explored, producing the same prefix $(v_1, \dots, v_k)$ and thus the same sub-problem at level $k + 1$. The search trees are isomorphic up to depth K . $\square$

6.2 Cost Analysis

Let P denote the cost of computing the shared prefix (iterating all valid $(v_1, \dots, v_K)$ bindings), let S_i denote the suffix cost for query Q_i , and let N denote the number of queries in the prefix group.

Without sharing:

$$C_{\text{independent}} = N \cdot (P + \bar{S}) + N \cdot C_{\text{trie}}$$

where C_{trie} is the per-query trie construction cost.

With cross-query optimizations:

$$C_{\text{shared}} = P + N \cdot \bar{S} + C_{\text{trie}} + C_{\text{coord}}$$

where C_{coord} captures the overhead of cross-query coordination: **TrieCache** lookup overhead, spool materialization cost from **CombineSharedComponentsRule** (Section 5.2), prefix binding materialization, and per-binding dispatch to suffix executors.

Savings:

$$\Delta C = (N - 1) \cdot P + (N - 1) \cdot C_{\text{trie}} - C_{\text{coord}}$$

The prefix cost P dominates when the prefix covers most of the variables, the common case for queries that differ only in their final projection, aggregation, or filter on non-join columns. For N identical triangle queries differing only in projection, $K = 3 = m$ (all variables shared), so $\bar{S} \approx 0$ and the savings approach $(N - 1) \cdot C_{\text{full_WCOJ}} - C_{\text{coord}}$. The coordination overhead C_{coord} grows as $O(|Q(D)|)$, proportional to the output size. Prefix bindings must be materialized and dispatched to suffix executors, so the net benefit depends on N being large enough for the $(N - 1) \cdot P$ savings to dominate.

7. Experimental Evaluation

7.1 Experimental Setup

We evaluate our system using a custom benchmark (`WCOJBenchmarkCli`) that compares four execution modes:

Mode	Description
baseline	Standard Calcite with binary hash joins, queries run sequentially
wcoj	WCOJ for each query, run sequentially (no multi-query optimization)
combine	<code>MULTI()</code> with WCOJ and trie caching (batched execution)
combine-share	<code>MULTI()</code> with WCOJ + trie caching + sub-expression sharing + prefix sharing

Graph generation. We construct synthetic directed graphs where a small fraction of vertices (default 5%) are highly connected “hubs” with many incoming and outgoing edges, while the remaining vertices have few connections. This hub-and-spoke structure mimics the skewed degree distributions found in real-world graphs (social networks, web graphs). Hubs create large intermediate results during binary joins because any two-hop path through a hub fans out widely. We also add direct edges between non-hub vertices to guarantee that triangles exist in the graph, not just paths through hubs.

Query workload. Triangle queries $Q_{\Delta}(a, b, c) \leftarrow R(a, b), S(b, c), T(c, a)$ with 5 projection variations, wrapped in `MULTI()` for batched modes.

Hardware and software. All experiments are run on an Apple M4 Pro (14 cores) with 48 GB RAM, OpenJDK 21.0.9, and JVM heap limited to 2 GB (`-Xmx2g`). Calcite version is 1.41.0-SNAPSHOT.

7.2 Results

Unless otherwise noted, timings are reported as the mean $\pm$ sample standard deviation over 10 iterations after warmup. The synthetic graph workload (Sections 7.2.1 to 7.2.3) uses 3 warmup rounds; the TPC-H workload (Section 7.2.4) uses 5 warmup rounds to account for the larger code paths generated by multi-table TPC-H queries (JIT compilation of the TPC-H WCOJ code path requires more invocations to stabilize than the single-table synthetic workload). All TPC-H shapes (triangle, 4-cycle, FK-triangle) use the same 5-warmup, 10-iteration protocol; the 4-cycle data was re-collected during revision to increase from 5 to 10 measured iterations. The synthetic workload consists of triangle queries $Q_{\Delta}(a, b, c) \leftarrow R(a, b), S(b, c), T(c, a)$ with 5 projection variations batched via `MULTI()`.

7.2.1 Scalability with Graph Size We fix the query batch size at $N = 5$ and vary graph size from 50 to 400 nodes, scaling edge counts proportionally with 5% hub nodes.

Table 2. Execution time (ms, mean $\pm$ sample std dev) and speedup vs. graph size, $N = 5$ triangle queries. Speedup over baseline shown in parentheses. [†]CV > 20%.

Graph	Triangles	Baseline	WCOJ	Combine	Combine-Share
50 / 300	333	86 ± 8	58 ± 3 (1.5x)	37 ± 3 (2.3x)	37 ± 2 (2.3x)
100 / 800	999	170 ± 12	156 ± 41 [†] (1.1x)	71 ± 7 (2.4x)	84 ± 9 (2.0x)
200 / 2000	1,854	265 ± 44	163 ± 11 (1.6x)	116 ± 15 (2.3x)	137 ± 24 (1.9x)
400 / 5000	3,525	772 ± 88	241 ± 31 (3.2x)	202 ± 24 (3.8x)	192 ± 17 (4.0x)

Analysis. Speedups grow consistently as graph size increases:

- *Baseline grows faster than WCOJ.* Baseline execution time grows from 86 ms at $|V| = 50$ to 772 ms at $|V| = 400$, a 9.0x increase for a 10.6x increase in triangle count (333 to 3,525). The widening gap reflects intermediate result explosion: the two-way join $R \bowtie S$ produces $O(|R| \cdot |S|/|V|)$ tuples, many subsequently eliminated by the third join. WCOJ avoids materializing these intermediates, so its growth tracks output size more closely (4.2x over the same range).
- *WCOJ scales sub-linearly.* Speedup over baseline increases from 1.5x at $|V| = 50$ to 3.2x at $|V| = 400$, demonstrating that the advantage compounds as the join graph becomes denser.
- *Combine provides consistent batching benefit.* The **Combine** operator with trie caching outperforms sequential WCOJ at all sizes, reaching 3.8x over baseline at $|V| = 400$.
- *Combine-Share achieves the peak speedup of 4.0x at $|V| = 400$.* The crossover from Combine (3.8x) occurs because sharing overhead is fixed while redundant-computation savings grow with graph density. At smaller sizes, the overhead dominates.

7.2.2 Multi-Query Speedup We fix the graph at $|V| = 200$, $|E| = 2000$ and vary the number of batched triangle queries from $N = 5$ to $N = 20$.

Table 3. Execution time (ms, mean ± sample std dev), speedup, and per-query cost vs. query batch size. Speedup over baseline in parentheses for batched modes (Combine, Combine-Share). WCOJ speedups are omitted because WCOJ executes queries sequentially (one at a time) and is not the primary comparison target for the batch-size experiment; its column serves as a non-batched reference point. Results marked with † have coefficient of variation $> 20\%$.

N	Baseline	WCOJ	Comb.	C-Share	/query: C	/query: CS
5	274 ± 33	182 ± 44 [†]	110 ± 9 (2.5x)	116 ± 12 (2.4x)	22	23
10	559 ± 176 [†]	297 ± 38	185 ± 34 (3.0x)	165 ± 23 (3.4x)	19	17
20	597 ± 109	426 ± 64	224 ± 16 (2.7x)	247 ± 55 [†] (2.4x)	11	12

Analysis. Per-query amortized cost for **Combine** drops from 22 ms at $N = 5$ to **11 ms** at $N = 20$, demonstrating sub-linear scaling as fixed costs (plan compilation, trie construction) are amortized

across the batch. We omit $N = 2$: both batched modes exhibit $CV > 50\%$ at that size due to JIT compilation effects.

Combine-Share peaks at $N = 10$ (3.4x over baseline, vs. 3.0x for plain Combine), where each of 5 query variations appears twice, giving prefix sharing genuine redundancy to eliminate. At $N = 20$, plain Combine regains the lead (2.7x vs. 2.4x). Section 7.2.3 explains this reversal.

7.2.3 Optimization Contributions The four modes form a natural incremental comparison: each mode adds one optimization layer, isolating its marginal contribution. We report results at two operating points that illustrate different regimes:

Table 4. Optimization contributions at $|V| = 400$, $N = 5$ (size-dominated) and $|V| = 200$, $N = 10$ (batch-dominated). “Marginal gain” shows the additional speedup each layer adds over the previous.

Mode	V=400, N=5	Marginal gain	V=200, N=10	Marginal gain
Baseline	772 ms	-	559 ms	-
+ WCOJ	241 ms (3.2x)	3.2x	297 ms (1.9x)	1.9x
+ Combine	202 ms (3.8x)	+0.6x	185 ms (3.0x)	+1.1x
+ Combine-Share	192 ms (4.0x)	+0.2x	165 ms (3.4x)	+0.4x

Analysis. The dominant optimization is the WCOJ algorithm itself, which eliminates the intermediate result explosion in binary joins on cyclic queries. Batched execution via **Combine** (with trie caching) provides the next increment by amortizing plan compilation and trie construction across the batch.

The contribution of Combine-Share over plain Combine depends on the operating regime. At $|V| = 400$, $N = 5$, the improvement is modest (202 ms $\rightarrow$ 192 ms, 1.05x). At $|V| = 200$, $N = 10$, it is more significant (185 ms $\rightarrow$ 165 ms, 1.12x), because $N = 10$ contains duplicate sub-queries (each of 5 variations appears twice), giving the sub-expression sharing and prefix-sharing rules genuine redundancy to eliminate.

The formal cost model (Section 6.2) predicts savings of $(N - 1) \cdot P - C_{\text{coord}}$. The experimental results confirm this structure: when the prefix cost P is large relative to the coordination overhead C_{coord} (larger graphs, moderate duplication), Combine-Share wins. When C_{coord} dominates, plain Combine is preferable. At $N = 20$ in Table 3, the prefix-sharing coordination overhead grows with output size while plain Combine benefits from trie caching alone.

7.2.4 TPC-H Cyclic Joins The synthetic graph benchmarks above use self-joins on a single edge table, a pattern that maximizes WCOJ’s advantage. To validate these results on realistic data, we construct cyclic join queries over the TPC-H schema (SF=0.01) and introduce a **combine-binary** mode (`MULTI()` batching with standard binary hash joins, no WCOJ) to isolate the contributions of WCOJ and batching.

Query design. Standard TPC-H queries are acyclic, but the schema naturally supports cyclic patterns. The `lineitem` table has multiple join keys (`l_orderkey`, `l_suppkey`, `l_partkey`), creating self-join triangles when joined on different key combinations:


```
-- Self-join triangle: orderkey-suppley-partkey cycle
SELECT l1.l_orderkey, l2.l_suppkey, l3.l_partkey
FROM lineitem l1, lineitem l2, lineitem l3
WHERE l1.l_orderkey = l2.l_orderkey    -- same order
      AND l2.l_suppkey = l3.l_suppkey   -- same supplier
      AND l3.l_partkey = l1.l_partkey   -- same part (closes cycle)
```

This finds lineitem triples linked by shared orders, suppliers, and parts, a supply-chain co-occurrence pattern. Binary joins face intermediate result explosion: the first join on `orderkey` produces $O(|L|^2/|O|)$ tuples ($\sim 4x$ fan-out per row), and the second join on `suppley` expands further (~ 600 items per supplier at $SF=0.01$). We also test a self-join 4-cycle:

```
-- Self-join 4-cycle: suppley-orderkey-partkey-orderkey
SELECT l1.l_orderkey, l2.l_suppkey, l3.l_partkey, l4.l_orderkey AS ok4
FROM lineitem l1, lineitem l2, lineitem l3, lineitem l4
WHERE l1.l_suppkey = l2.l_suppkey    -- L1-L2: same supplier
      AND l2.l_orderkey = l3.l_orderkey -- L2-L3: same order
      AND l3.l_partkey = l4.l_partkey  -- L3-L4: same part
      AND l4.l_orderkey = l1.l_orderkey -- L4-L1: same order (closes cycle)
```

Note that `orderkey` appears in two distinct join predicates (L2–L3 and L4–L1), each linking a different pair of lineitem copies (valid because the cycle traverses four copies through four distinct edges). We additionally test a foreign-key triangle (`lineitem` $\bowtie$ `partsupp` $\bowtie$ `supplier`) and two FK-based cyclic queries to characterize WCOJ’s failure modes.

Table 5. TPC-H cyclic join performance ($SF=0.01$, $N = 5$ projection variations, 10 iterations, 5 warmup rounds). Mean $\pm$ sample std dev in ms. Speedup over baseline in parentheses. [†]High variance ($CV > 20\%$). *Out of memory with 4 GB heap; the 4-way binary join intermediate exceeds available memory.

Query	Rows	Base	C-Bin	WCOJ	Comb.	C-Share
SJ $\triangle$	2.9M	6102 $\pm$ 1349 [†]	3546 $\pm$ 226 (1.7x)	2998 $\pm$ 425 (2.0x)	3884 $\pm$ 506 (1.6x)	2511 $\pm$ 82 (2.4x)
SJ $\square$	6.0M	128311 $\pm$ 30538 [†]	OOM*	40772 $\pm$ 534 (3.1x)	43889 $\pm$ 421 (2.9x)	46343 $\pm$ 1499 (2.8x)
FK $\triangle$	301K	928 $\pm$ 109	599 $\pm$ 49 (1.5x)	881 $\pm$ 255 [†] ($\sim 1.0x$)	741 $\pm$ 148 (1.3x)	573 $\pm$ 63 (1.6x)

Table 6. TPC-H FK queries where WCOJ *underperforms* baseline ($SF=0.01$, $N = 5$, 10 iterations). Mean $\pm$ sample std dev in ms.

Query	Rows	Base (ms)	WCOJ (ms)	Ratio
FK $\square$ (c-o-l-s)	11,665	860 $\pm$ 30	1,314 $\pm$ 41	1.5x slower
FK $\diamond$ (c-o-l-s-n)	11,665	2,237 $\pm$ 25	7,950 $\pm$ 107	3.6x slower

Combine and Combine-Share modes show the same regression on these queries (1,350 ms and 1,359 ms for FK rectangle; 8,287 ms and 8,471 ms for FK diamond); batching does not recover

the WCOJ penalty when the join algorithm itself is the bottleneck. Full data is archived in docs/benchmark-results/tpch-wcoj-sf001.csv.

[**TODO: Profile the FK regressions to confirm the per-level intersection cost explanation. Try disk-spillable spools to see if GC pressure is a factor. See future work item (5).**]

Analysis. Baseline measurements for the triangle and 4-cycle exhibit high variance ($CV = 22\text{--}24\%$, marked $\dagger$), driven by GC pressure and OS page cache state on multi-second queries; all TPC-H speedup ratios should be interpreted as estimates with $\sim 20\%$ uncertainty.

When WCOJ wins. The self-join queries demonstrate clear WCOJ benefits: the 4-cycle achieves **3.1x** speedup and the triangle **2.0x**, driven by intermediate result explosion in binary join plans. The 4-cycle is particularly telling: binary joins run out of memory at 4 GB heap (marked OOM in Table 5), while WCOJ completes in 41 seconds, a qualitative difference in feasibility. The combine-binary column confirms that batch amortization alone accounts for much of the gain (1.7x on the self-join triangle), with WCOJ providing the remainder. On the FK triangle, WCOJ (881 ± 255 ms) is not statistically distinguishable from baseline (928 ± 109 ms); the benefit comes entirely from batching and sharing (Combine-Share: 573 ms).

When WCOJ loses. The FK rectangle and diamond queries (Table 6) show WCOJ 1.5x–3.6x *slower* than binary joins. The critical difference is join selectivity: the FK queries close their cycles through **nationkey** (25 distinct values), creating a highly selective closing predicate that binary hash joins handle efficiently. WCOJ pays $O(\text{fan-out})$ intersection cost per variable at each level even when most candidates are eliminated; this is the search cost omitted from the lower-bound model in Section 3.5. The self-join queries, by contrast, close through high-cardinality keys (300K–6M output rows), making WCOJ’s avoidance of intermediate materialization the dominant effect. On FK joins, combine-binary (599 ms) is *faster* than combine-WCOJ (741 ms), confirming that WCOJ actively hurts when binary joins are already efficient.

Combine overhead. Plain Combine is 30% slower than sequential WCOJ on the self-join triangle (3,884 ms vs. 2,998 ms) and 8% slower on the 4-cycle. This stems from forced result materialization (EnumerableCombine calls `.toList()` on each child, materializing ~ 14.5 M row objects for the triangle) and monolithic code generation that prevents per-query JIT specialization. Combine-Share (2,511 ms) recovers on the triangle by computing the shared prefix once across 5 variants, but not on the 4-cycle (46,343 ms), where coordination overhead on 6M output rows exceeds the savings.

7.2.5 Limitations The experiments are subject to several constraints that bound the scope of the claims:

- **Scale factor.** TPC-H experiments use $SF=0.01$ (~ 60 MB). At larger scale factors, both the WCOJ speedups on self-join queries and the regressions on FK queries are expected to grow, but the crossover point between beneficial and harmful WCOJ application has not been characterized beyond $SF=0.01$.
- **In-memory spools.** Sub-expression sharing via spooling requires holding materialized intermediate results in memory. The current implementation does not spill to disk; queries that exceed heap will OOM rather than degrade gracefully.
- **Single machine.** All experiments run on a single M4 Pro. The framework’s interaction with distributed query processing (e.g., Calcite over Flink or Trino) has not been evaluated.
- **Homogeneous batch structure.** The workload uses N projection variants over the same join structure. Workloads with structurally diverse queries across a batch would see less

benefit from sub-expression sharing and prefix execution.

8. Conclusion

We have presented a unified framework within Apache Calcite that combines worst-case optimal join algorithms with multi-query optimization. Our system introduces the **Combine** relational operator and `MULTI()` SQL syntax for declarative multi-query batching, a hash-based WCOJ implementation with multi-level trie indexing, and three cross-query optimizations: identity-based trie caching (implicit in **Combine**), sub-expression sharing via frequency-aware spooling, and shared-prefix execution through join-variable fingerprinting.

The key insight underlying our approach is that WCOJ’s variable-at-a-time execution creates natural sharing opportunities that do not exist in binary join plans: trie structures can be shared, identical backtracking search trees can be traversed once, and a shared variable prefix can be factored out across queries. The **Combine** operator with trie caching achieves up to **4.0x** speedup on synthetic cyclic queries, with per-query amortized cost dropping 9.5x as batch size grows. On TPC-H data, WCOJ enables queries that binary joins cannot complete within memory limits, and achieves **3.1x** speedup on self-join patterns where intermediate results dominate.

The experimental results point toward a broader lesson. Our WCOJ implementation is not universally beneficial: FK-based cycles where binary joins already handle selective closing predicates efficiently expose a cost model gap that causes regressions of 1.5x–3.6x. The real value of the **Combine** framework is providing a *platform for selective optimization*: the optimizer can route cyclic sub-queries to WCOJ where it helps while leaving acyclic or low-fanout components to binary joins, and the **Combine** operator amortizes fixed costs across the batch regardless of which strategy is chosen. Future work item (4), hybrid join strategies within a batch, directly addresses this.

Our work does not address the general MQO selection problem [10]; it exploits the specific structure of WCOJ execution to identify sharing opportunities within batches of cyclic join queries. This structural approach is complementary to selection-based MQO and could be integrated with it for mixed workloads. All contributions are implemented as modular extensions to Apache Calcite, preserving backward compatibility and enabling adoption by the systems built on the Calcite framework.

Future work. Several directions remain open: (1) reducing the coordination overhead C_{coord} of prefix sharing by passing prefix bindings via shared memory rather than materializing them as intermediate result sets, which would extend Combine-Share’s advantage to larger batch sizes where materialization cost currently dominates; (2) exploring adaptive variable ordering that considers both single-query and cross-query optimization objectives; (3) integrating with Calcite’s materialized view subsystem for persistent cross-batch sharing; (4) hybrid join strategies within a batch, using WCOJ for cyclic components and binary joins for acyclic components, as motivated by the DpHyp relationship in Section 3.2; and (5) disk-spillable spools, allowing intermediate results to exceed heap memory and degrade gracefully rather than OOM. The current in-memory-only spool design limits scalability at larger scale factors and wider projections; integrating with Calcite’s existing disk-backed spool infrastructure would remove this constraint.

References

- [1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, “Access path selection in a relational database management system,” in *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, 1979, pp. 23–34. doi: 10.1145/582095.582099
- [2] A. Atserias, M. Grohe, and D. Marx, “Size bounds and query plans for relational joins,” *SIAM Journal on Computing*, vol. 42, no. 4, pp. 1737–1767, 2013. doi: 10.1137/110859440
- [3] H. Q. Ngo, E. Porat, C. Re, and A. Rudra, “Worst-case optimal join algorithms,” *Journal of the ACM*, vol. 65, no. 3, pp. 1–40, 2018. doi: 10.1145/3180143
- [4] T. L. Veldhuizen, “Leapfrog Triejoin: A simple, worst-case optimal join algorithm,” in *Proc. 17th International Conference on Database Theory (ICDT)*, Athens, Greece, 2014, pp. 96–106. Available: https://openproceedings.org/ICDT/2014/paper_13.pdf
- [5] M. Freitag, M. Bandle, T. Schmidt, A. Kemper, and T. Neumann, “Adopting worst-case optimal joins in relational database systems,” *Proceedings of the VLDB Endowment*, vol. 13, no. 11, pp. 1891–1904, 2020. doi: 10.14778/3407790.3407797
- [6] T. K. Sellis, “Multiple-query optimization,” *ACM Transactions on Database Systems*, vol. 13, no. 1, pp. 23–52, 1988. doi: 10.1145/42201.42203
- [7] T. Kathuria and S. Sudarshan, “Efficient and provable multi-query optimization,” in *Proc. 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*, 2017, pp. 53–67. doi: 10.1145/3034786.3034792
- [8] P. Roy, S. Seshadri, S. Sudarshan, and S. Bhowe, “Efficient and extensible algorithms for multi query optimization,” *ACM SIGMOD Record*, vol. 29, no. 2, pp. 249–260, 2000. doi: 10.1145/335191.335419
- [9] E. Begoli, J. Camacho-Rodriguez, J. Hyde, M. J. Mior, and D. Lemire, “Apache Calcite: A foundational framework for optimized query processing over heterogeneous data sources,” in *Proc. 2018 ACM International Conference on Management of Data (SIGMOD)*, Houston, TX, 2018, pp. 221–230. doi: 10.1145/3183713.3190662
- [10] S. Zinchenko and D. Ponomaryov, “The selection problem in multi-query optimization: A comprehensive survey,” *arXiv preprint arXiv:2412.11828*, 2025. doi: 10.48550/arXiv.2412.11828
- [11] Y. Tu, M. Eslami, Z. Xu, and H. Charkhgard, “Multi-query optimization revisited: A full-query algebraic method,” in *Proc. IEEE International Conference on Big Data*, 2022, pp. 252–261. doi: 10.1109/BigData55660.2022.10020338
- [12] B. Gurumurthy, V. R. Bidarkar, D. Briones, T. Pionteck, and G. Saake, “Exploiting shared sub-expression and materialized view reuse for multi-query optimization,” *Information Systems Frontiers*, 2024. doi: 10.1007/s10796-024-10506-w
- [13] Y. Tian, “Query optimization in the wild: Realities and trends,” *arXiv preprint arXiv:2510.20082*, 2025. doi: 10.48550/arXiv.2510.20082
- [14] C. R. Aberger, A. Lamb, S. Tu, A. Notzli, K. Olukotun, and C. Re, “EmptyHeaded: A relational engine for graph processing,” *ACM Transactions on Database Systems*, vol. 42, no. 4, pp. 20:1–20:44, 2017. doi: 10.1145/3129246

- [15] Y. R. Wang, M. Willsey, and D. Suciu, “Free Join: Unifying worst-case optimal and traditional joins,” *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, Article 150, 2023. doi: 10.1145/3589295
- [16] P. Michiardi, D. Carra, and S. Migliorini, “Cache-based multi-query optimization for data-intensive scalable computing frameworks,” *Information Systems Frontiers*, vol. 23, pp. 35–51, 2021. doi: 10.1007/s10796-020-09995-2
- [17] M. Schönberger, I. Trummer, and W. Mauerer, “Large-scale multiple query optimisation with incremental quantum(-inspired) annealing,” *Proceedings of the ACM on Management of Data*, vol. 3, no. 4, Article 253, pp. 253:1–253:25, 2025. doi: 10.1145/3749171
- [18] H. Q. Ngo, C. Re, and A. Rudra, “Skew strikes back: New developments in the theory of join algorithms,” *ACM SIGMOD Record*, vol. 42, no. 4, pp. 5–16, 2013. doi: 10.1145/2590989.2590991
- [19] G. Graefe and W. J. McKenna, “The Volcano optimizer generator: Extensibility and efficient search,” in *Proc. 9th IEEE International Conference on Data Engineering (ICDE)*, Vienna, Austria, 1993, pp. 209–218. doi: 10.1109/ICDE.1993.344061
- [20] G. Graefe, “The Cascades framework for query optimization,” *IEEE Data Engineering Bulletin*, vol. 18, no. 3, pp. 19–29, 1995.
- [21] G. Gottlob, N. Leone, and F. Scarcello, “Hypertree decompositions and tractable queries,” *Journal of Computer and System Sciences*, vol. 64, no. 3, pp. 579–627, 2002. doi: 10.1006/jcss.2001.1809
- [22] S. Harizopoulos, V. Shkapenyuk, and A. Ailamaki, “QPipe: A simultaneously pipelined relational query engine,” in *Proc. 2005 ACM SIGMOD International Conference on Management of Data*, 2005, pp. 383–394. doi: 10.1145/1066157.1066201
- [23] S. Finkelstein, “Common expression analysis in database applications,” in *Proc. 1982 ACM SIGMOD International Conference on Management of Data*, 1982, pp. 235–245. doi: 10.1145/582353.582400
- [24] H. Q. Ngo, “Worst-case optimal join algorithms: Techniques, results, and open problems,” in *Proc. 37th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*, 2018, pp. 111–127. doi: 10.1145/3196959.3196990
- [25] A. Jindal, K. Karanasos, S. Rao, and H. Patel, “Selecting subexpressions to materialize at datacenter scale,” *Proceedings of the VLDB Endowment*, vol. 11, no. 7, pp. 800–812, 2018. doi: 10.14778/3192965.3192971
- [26] N. Bruno, J. Debrod, C. Song, and W. Zheng, “Computation reuse via fusion in Amazon Athena,” in *Proc. 38th IEEE International Conference on Data Engineering (ICDE)*, 2022, pp. 1756–1767. doi: 10.1109/ICDE53745.2022.00166
- [27] A. Roy, A. Jindal, P. Gomatam, X. Ouyang, A. Gosalia, N. Ravi, S. Mann, and P. Jain, “SparkCruise: Workload optimization in managed Spark clusters at Microsoft,” *Proceedings of the VLDB Endowment*, vol. 14, no. 12, pp. 3122–3134, 2021. doi: 10.14778/3476311.3476388
- [28] G. Moerkotte and T. Neumann, “Analysis of two existing and one new dynamic programming algorithm for the generation of optimal bushy join trees without cross products,” in *Proc. 32nd International Conference on Very Large Data Bases (VLDB)*, 2006, pp. 930–941.